Artificial Intelligence

Tut/Lab 9: Reinforcement Learning

COSC2129

Outline

- Assignment 2
- Assignment 3

- Recap: Planning, MDPs

- RL: highlights

- Q&A

Artificial Intelligence – Road map

8 Markov Decision Processes

9 Reinforcement Learning

10 Bayesian Reasoning

11 Review + AI Philosophy + Course recap (week 12)

Assignment 2

- Most of students could not finish all the questions
- Some students could finish only a few questions
- Solution will be released
- Marks will be released early next week.

Assignment 3

- Specification and code have been released
- Follow the instruction to work on solutions

- Grouping:
  - 7 ungroup students => G7(3) & G8(3), G15(1)

- Strategy: work on easy questions first, ask/discuss with others (not to copy others' solution!).

- Questions?

Recap: Planning – MDPs - RL

- Planning: Week 6
- MDPs: Week 8
- RL: This week

Activity – group discussion

- How did we “evolve” from automated planning to MDPs, and to Reinforcement learning?
- In other words, what is the difference between those topics in AI?

Recap

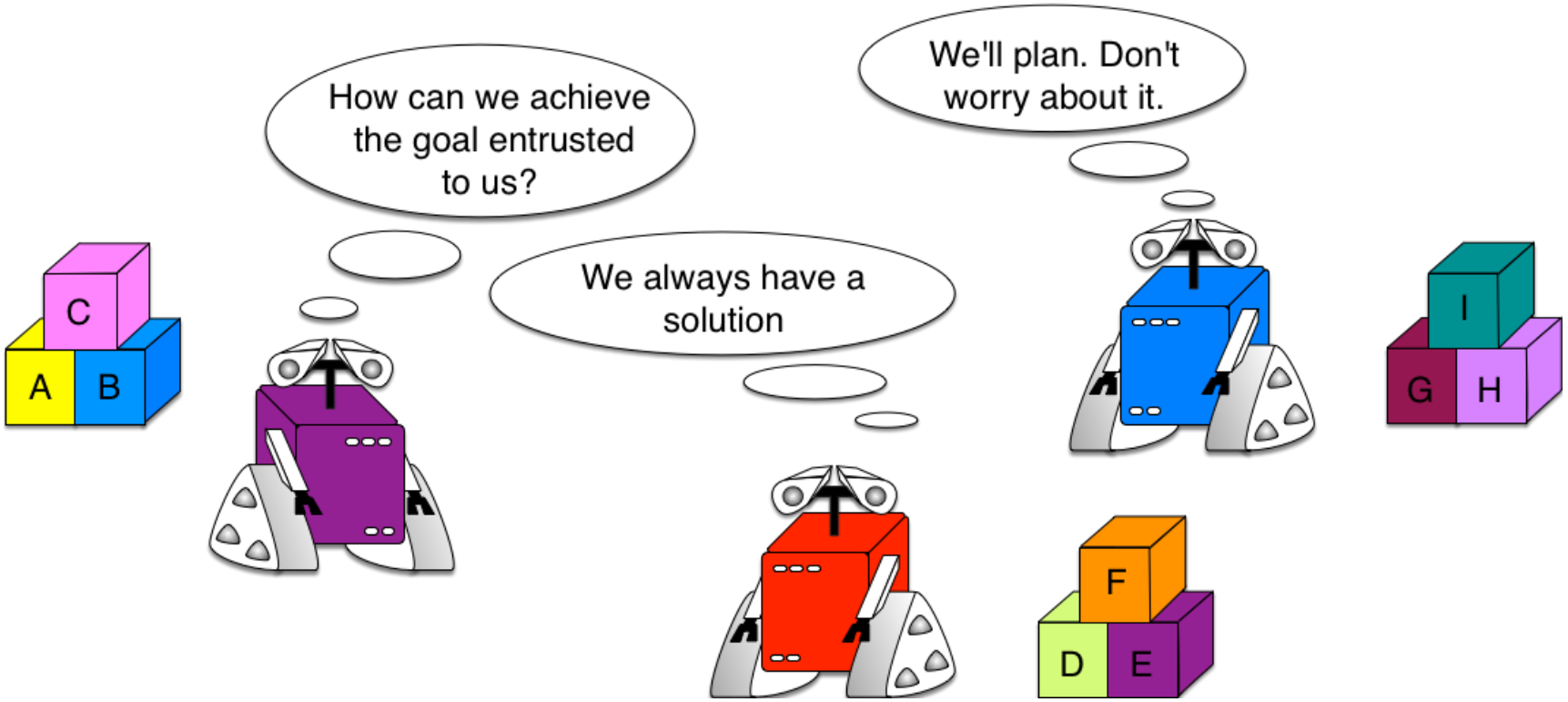
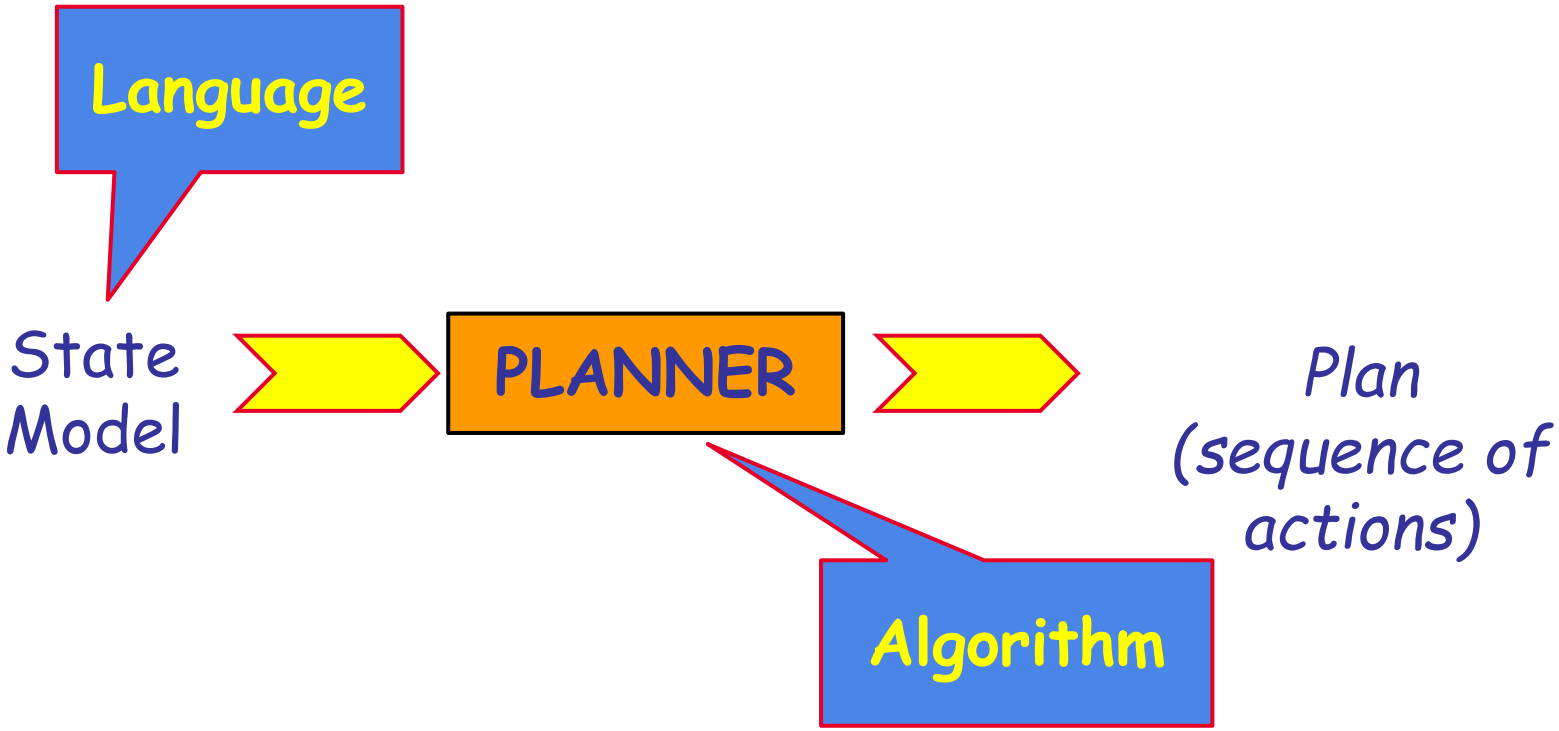
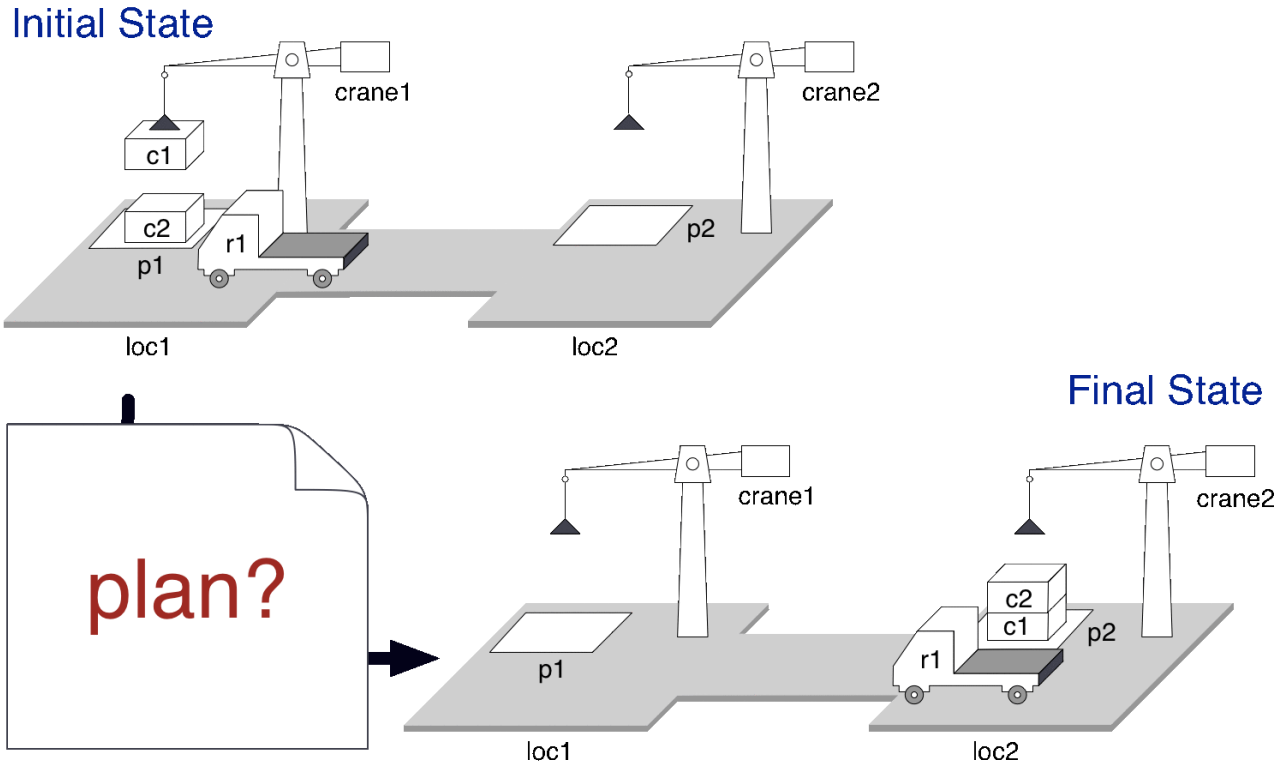


Fig. Source

Week 6: Classical Planning

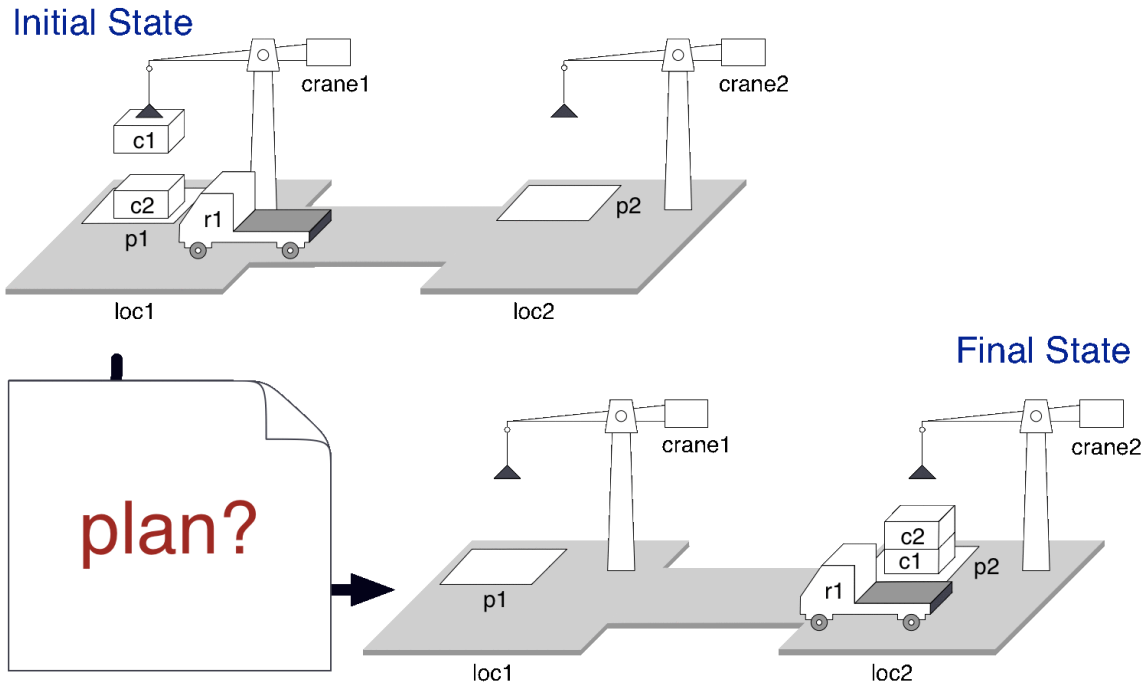


Automated planning



- Plan: Sequence of actions to accomplish certain task (goal)
- Solution for a planning problem: an action sequence that, when executing in the initial state, results in a state that satisfies the goal.

Automated planning



- The environment is completely observable, deterministics
- The solution plans are sequential
- Planning does not interleave action execution
- …

Source

MDP: Markov Decision Processes

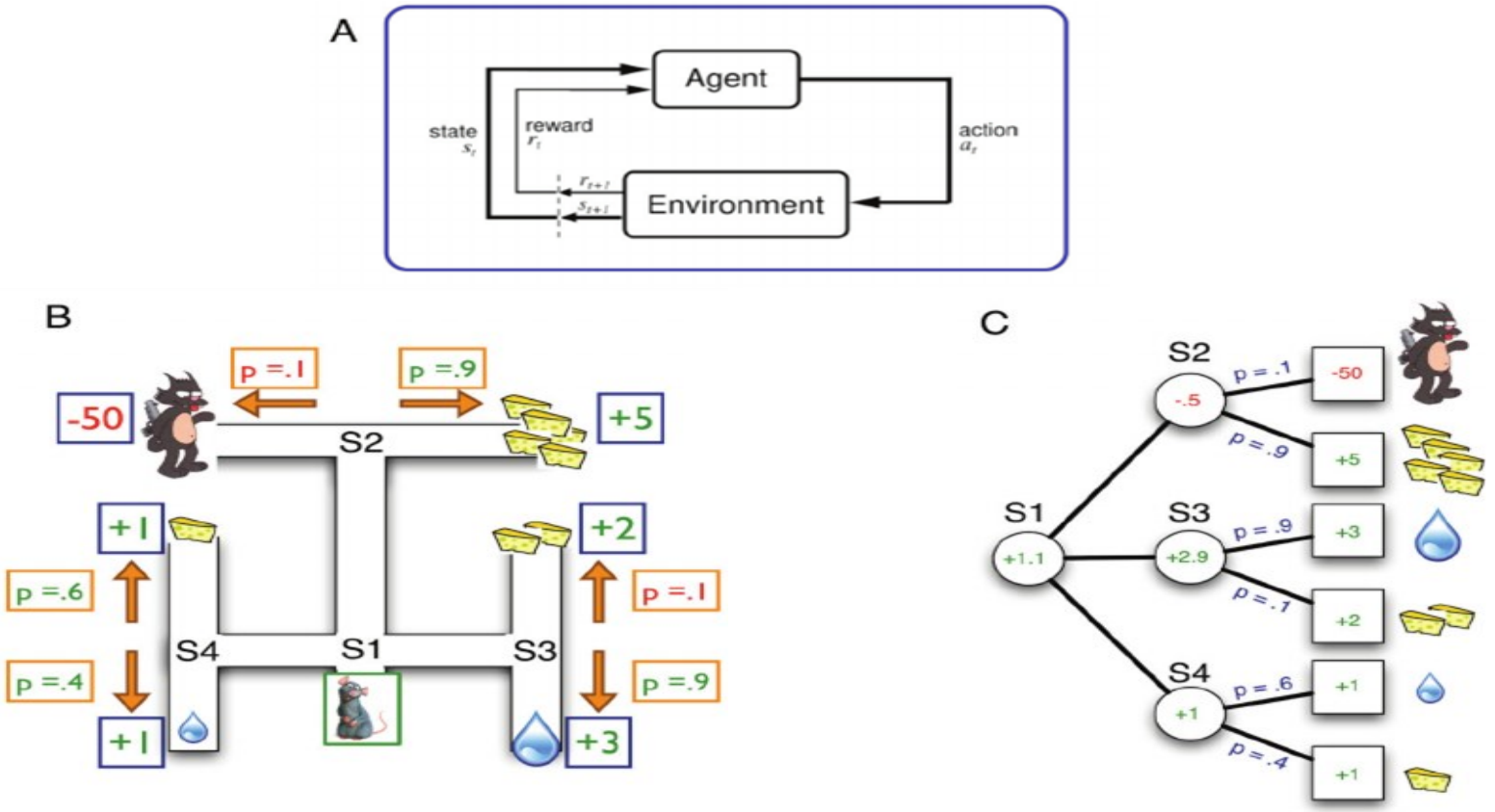
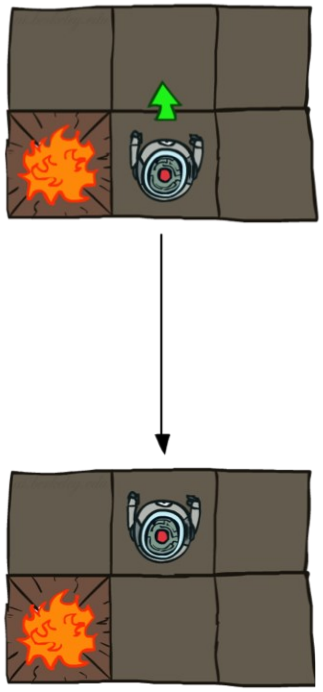


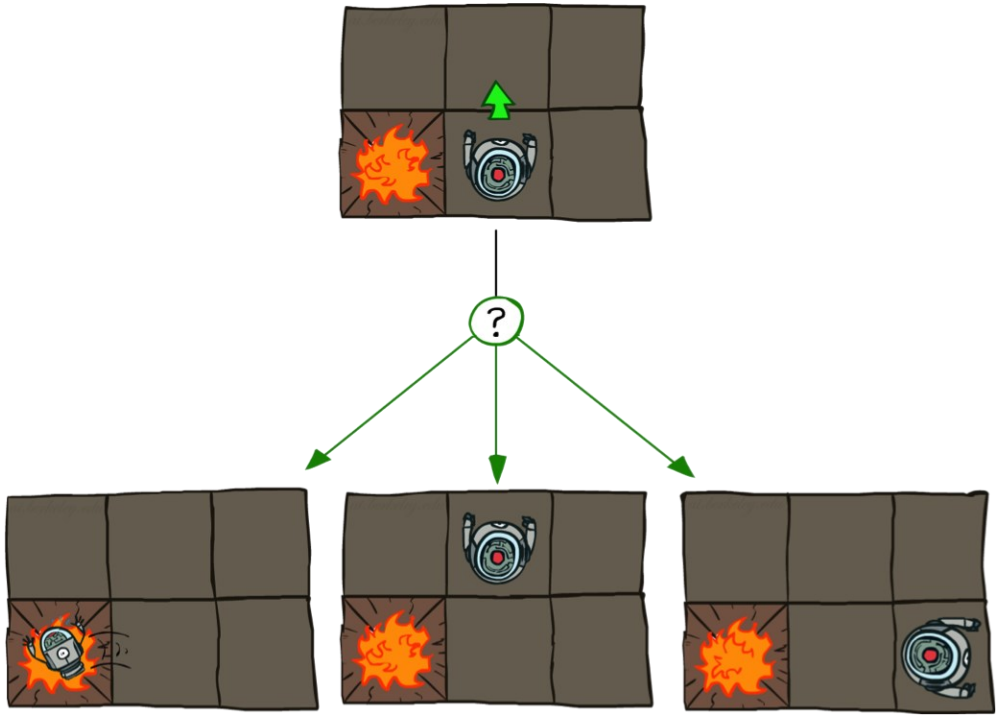
Image from: [A Primer on Reinforcement Learning in the Brain: Psychological, Computational, and Neural Perspectives]

GridWorld Actions

Deterministic Grid World



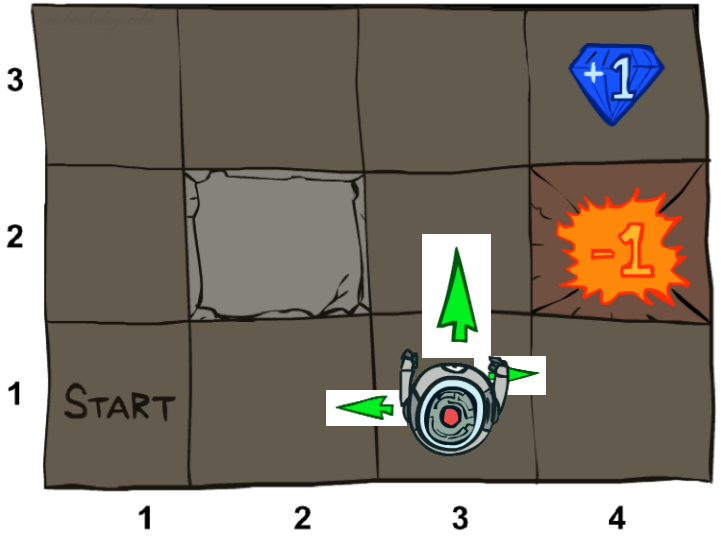
Stochastic Grid World



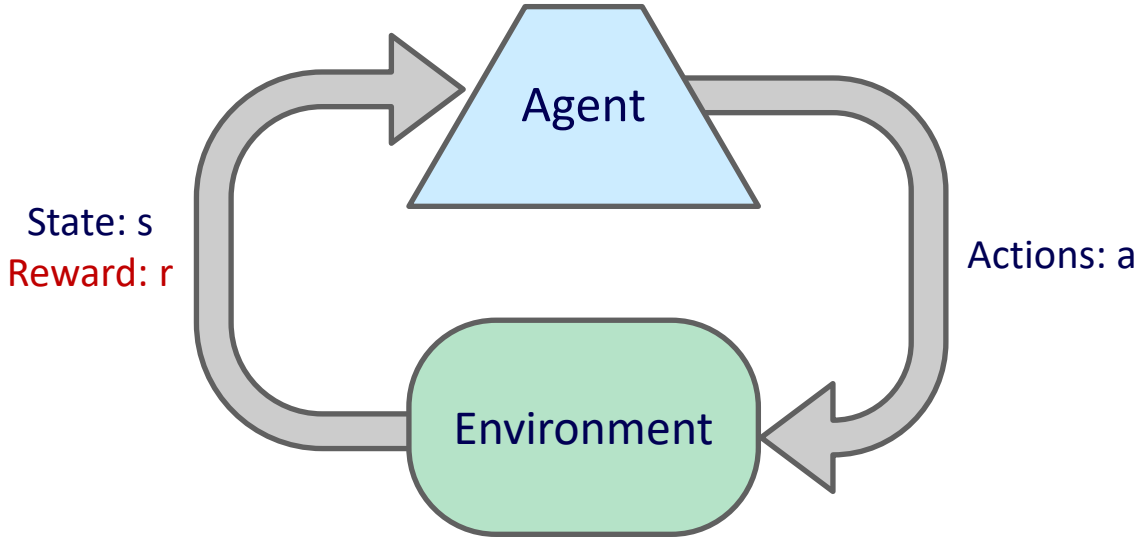
Markov Decision Processes (MDP)

An MDP is a tuple M = (S, A, T, R, s0, F):

1. A set of **states** s ∈ S:
   locations, clean, fuel, broken, ..
2. A set of **actions** a ∈ A:
   move to, suck, pick, drop, ...
3. A **transition function** T ("the model"):
   T(s, a, s'): Probability that a
   in s leads to s', i.e., P(s'| s, a)
   pick action succeeds 90%
4. A **reward function** R(s): value of state s.
   clean living room gives +10
5. A **start state** s0 ∈ S.
   robot initial location, etc..
6. Possibly **terminal states** F ⊆ S

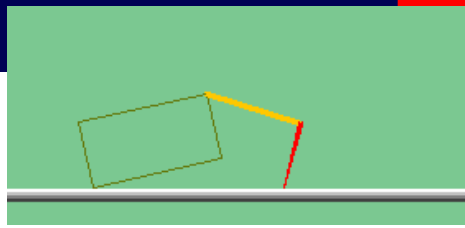


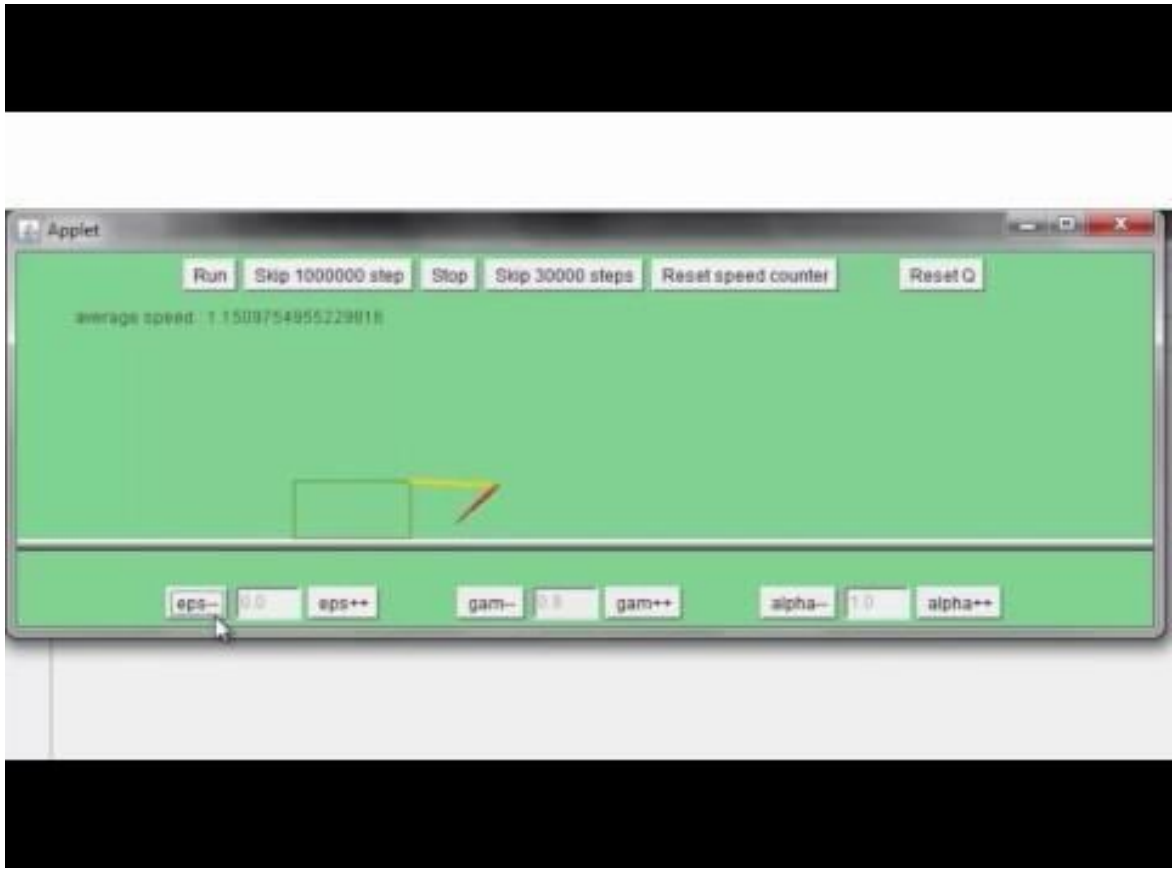
Reinforcement Learning



- Basic idea:
  - Receive feedback in the form of rewards
  - Agent's utility is defined by the reward function
  - Must (learn to) act so as to maximize expected rewards
  - All learning is based on observed samples of outcomes!

The Crawler!

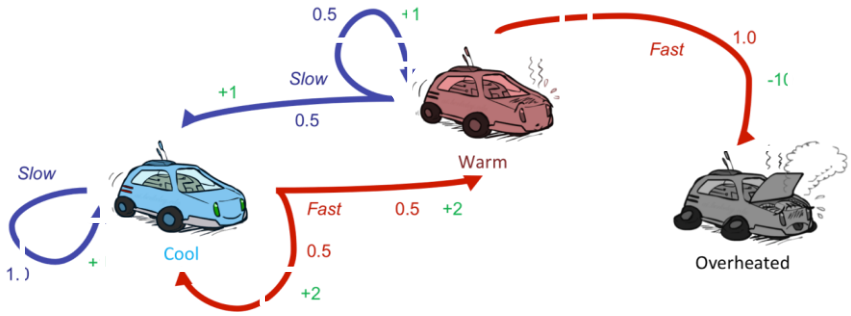




[Demo: Crawler Bot (L10D1)] [You, in Project 3]

Reinforcement Learning

- Still assume a Markov decision process (MDP):
  - A set of states s ∈ S
  - A set of actions (per state) A
  - A model T(s,a,s') = P(s' | s, a)
  - A reward function R(s,a,s')
- Still looking for a policy π(s)



- New twist: don't know T or R
  - I.e. we don't know which states are good or what the actions do
  - Must actually try actions and states out to learn

Model-based learning

- Model-Based Idea:
  - Learn an approximate model based on experiences
  - Solve for values as if the learned model were correct

- Step 1: Learn empirical MDP model
  - Count outcomes s' for each s, a
  - Normalize to give an estimate of $\hat{T}(s, a, s')$
  - Discover each $\hat{R}(s, a, s')$ when we experience (s, a, s')

- Step 2: Solve the learned MDP
  - For example, use value iteration, as before, to estimate value U(s) (or V(s))

How to estimate T(s,a,s')?

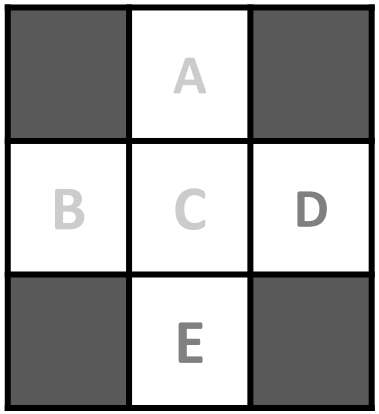
$$T(s,a,s') = \frac{number\ of\ times\ we\ seen\ (s,a,s')}{number\ of\ times\ we\ seen\ (s,a)}$$

E.g., if we have outcomes (C,east,D), (C,east,A), (C,east, D), then:

$$T(C,east,D) = \frac{2}{3}$$

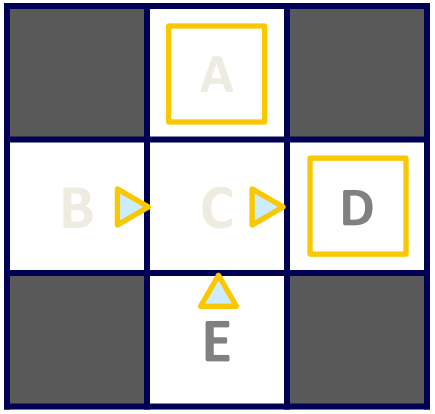
$$T(C,east,A) = \frac{1}{3}$$

$$T(C,east,E) = \frac{0}{3}, T(C,east,B) = \frac{0}{3}$$



Example: Model-Based Learning

Input Policy π



Assume: γ = 1

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$\hat{T}(s, a, s')$

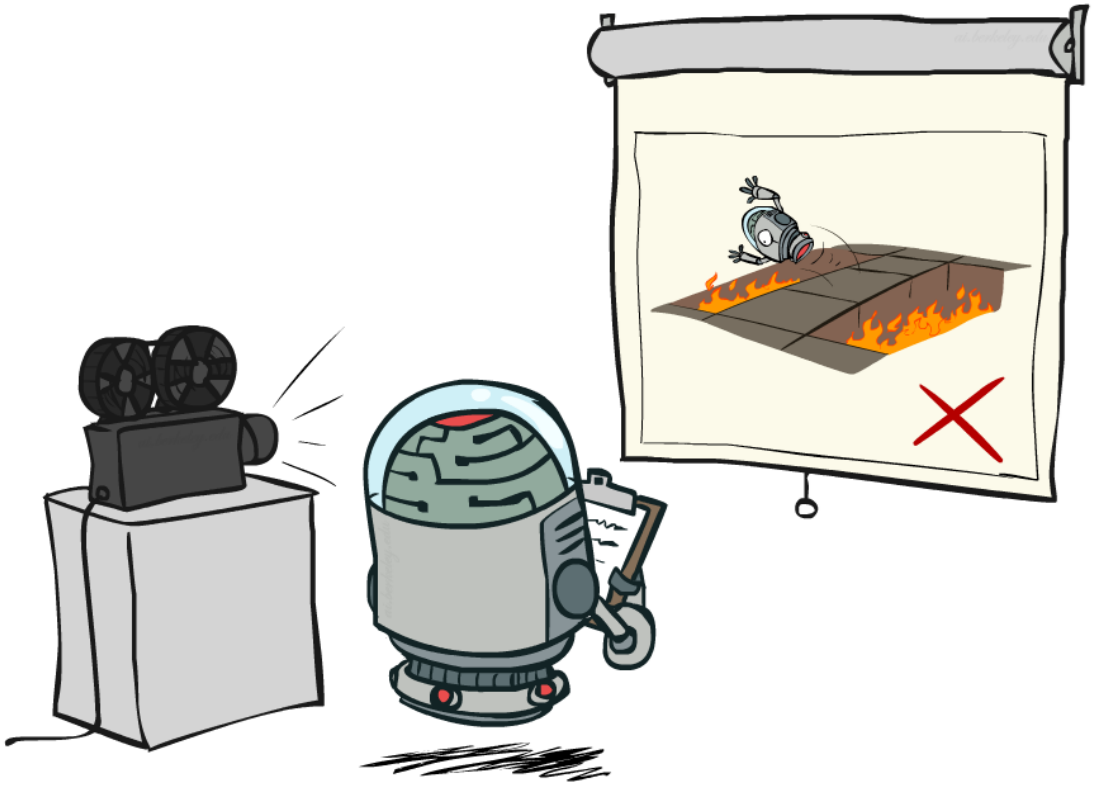
T(B, east, C) = 1.00
T(C, east, D) = 0.75
T(C, east, A) = 0.25
...

$\hat{R}(s, a, s')$

R(B, east, C) = -1
R(C, east, D) = -1
R(D, exit, x) = +10
...

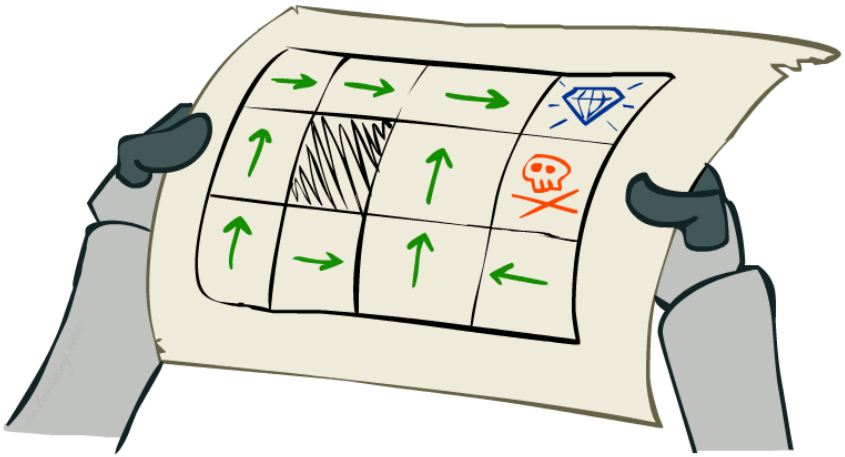
Model free

Passive Reinforcement Learning



Passive Reinforcement Learning

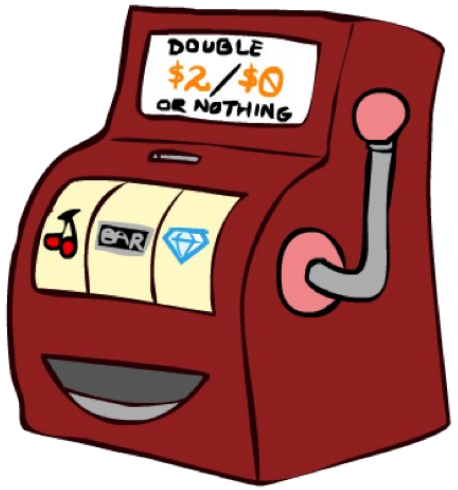
- Simplified task: policy evaluation
  - Input: a fixed policy π(s)
  - You don't know the transitions T(s,a,s')
  - You don't know the rewards R(s,a,s')
  - Goal: learn the state values



- In this case:
  - Learner is "along for the ride"
  - No choice about what actions to take
  - Just execute the policy and learn from experience
  - This is NOT offline planning! You actually take actions in the world.

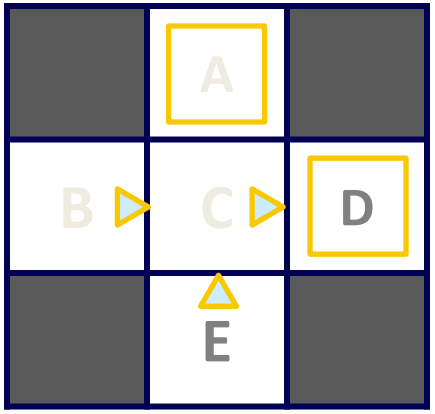
Direct Evaluation

- Goal: Compute values for each state under π
- Idea: Average together observed sample values
  - Act according to π.
  - Every time you visit a state, write down what the sum of discounted rewards turned out to be.
  - Average those samples.
- This is called direct evaluation



Example: Direct Evaluation

Input Policy π



Assume: γ = 1

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

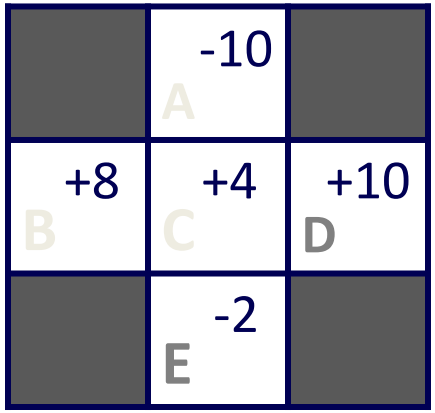
Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

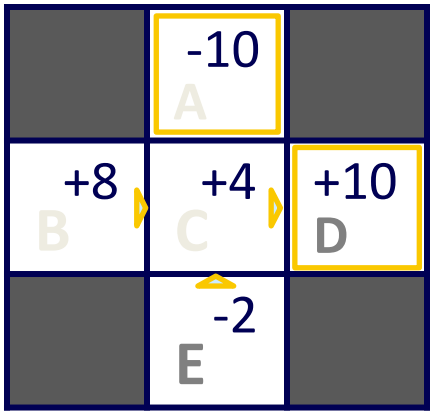
Output Values



Problems with Direct Evaluation

- What's good about direct evaluation?
  - It's easy to understand
  - It doesn't require any knowledge of T, R
  - It eventually computes the correct average values, using just sample transitions
- What bad about it?
  - It wastes information about state connections
  - Each state must be learned separately
  - So, it takes a long time to learn

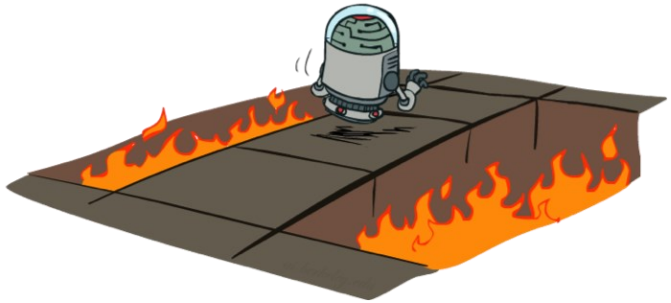
Output Values



If B and E both go to C under this policy, how can their values be different?

Active Reinforcement Learning

- Full reinforcement learning: optimal policies (like value iteration)
  - You don't know the transitions T(s,a,s')
  - You don't know the rewards R(s,a,s')
  - You choose the actions now
  - Goal: learn the optimal policy / values



- In this case:
  - Learner makes choices!
  - Fundamental tradeoff: exploration vs. exploitation
  - This is NOT offline planning! You actually take actions in the world and find out what happens…

Q-Value Iteration

- Value iteration: find successive (depth-limited) values
  - Start with $V_0(s) = 0$, which we know is right
  - Given V_k, calculate the depth k+1 values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

- But Q-values are more useful, so compute them instead
  - Start with $Q_0(s,a) = 0$, which we know is right
  - Given Q_k, calculate the depth k+1 q-values for all q-states:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

Q-Learning

- Q-Learning: sample-based Q-value iteration

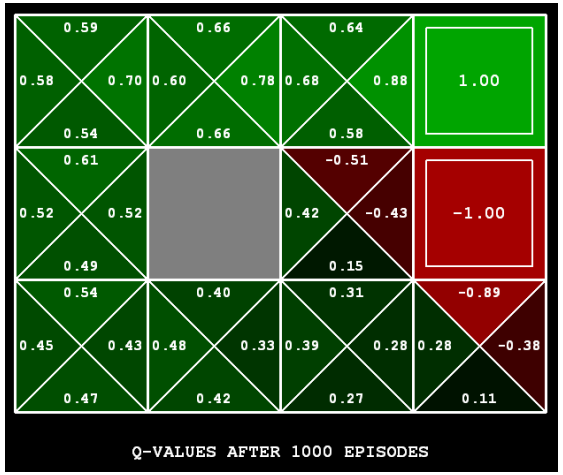
$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

- Learn Q(s,a) values as you go
  - Receive a sample (s,a,s',r)
  - Consider your old estimate: $Q(s,a)$
  - Consider your new sample estimate:

$$sample = R(s,a,s') + \gamma \max_{a'} Q(s',a')$$

  - Incorporate the new estimate into a running average:

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + (\alpha)\,[sample]$$



[Demo: Q-learning – gridworld (L10D2)]
[Demo: Q-learning – crawler (L10D3)]

Hand-on Practice

- AI22 Tut09

Question?